

PulseBuild

Technical Architecture v1.0

Implementation blueprint for Cursor, engineering modules, state management, data contracts, simulation, agent services, testing, and build phases.

Companion document to: PulseBuild Source of Truth v2.0 and PulseBuild Cognitive & Reasoning Specification.

Document role: This architecture document explains how to organize the codebase and implement the MVP safely. It does not replace the product or cognitive specification. It converts them into an engineering plan Cursor can follow.

Table of Contents

1. Architecture Purpose and Engineering Principles
2. Recommended MVP Stack
3. Repository and Folder Structure
4. System Runtime Architecture
5. Domain Model and TypeScript Contracts
6. Seeded Data Strategy
7. Reactive Engine Services
8. Agent Orchestration Layer
9. State Management Architecture
10. User Interface Architecture
11. Event Bus and Timeline Logging
12. Simulation Engine Architecture
13. Approval and Verification Workflow
14. Chat and Voice Integration
15. API and Persistence Strategy
16. Security, Guardrails, and Scope Boundaries
17. Testing Strategy
18. Cursor Build Workflow
19. Phase-by-Phase Implementation Plan
20. Definition of Done
21. Cursor Starter Prompt

1. Architecture Purpose and Engineering Principles

The purpose of this document is to translate PulseBuild from product vision and agent cognition into a buildable technical architecture. Cursor should use this document to scaffold the application, create services, define interfaces, isolate business logic, and avoid mixing agent reasoning with UI components.

Core engineering principle

PulseBuild must be deterministic-first and agent-assisted. Deterministic services calculate project state, priority, health, drift, recommendations, approval requirements, verification status, and audit events. The language model explains those results and helps users interact with the system. It must not become the source of truth for calculations.

Non-negotiable technical rules

- Keep domain logic outside React components.
- Keep simulation logic separate from the real reactive engine.
- Every signal, recommendation, approval, rejection, and verification must be traceable through an audit timeline.
- Never allow the chat layer to invent project state. It can read project state and explain it, but it cannot create facts without a structured user action.
- Every service must accept typed inputs and return typed outputs.
- Every high-impact action must go through Approval Engine before execution simulation.
- Every executed action must go through Verification Engine before being marked resolved.
- Cursor must build in phases. No future integrations, authentication, or enterprise features should be implemented in the MVP unless explicitly approved.

2. Recommended MVP Stack

This stack is designed for speed, clarity, and compatibility with Cursor. It supports a polished frontend, deterministic service architecture, seeded data, and optional future migration to a backend.

Layer	Recommended choice	Reason
Framework	Next.js with TypeScript	Strong routing, component structure, and future API route support.
UI	React components + Tailwind CSS	Fast styling, reusable components, polished command-center layout.
State	Zustand or React Context for MVP	Simple global state for simulation, project state, signals, recommendations, approvals.
Data	Seeded TypeScript JSON files	Fits MVP scope; avoids database setup before logic is proven.
Charts	Recharts or lightweight SVG components	Health, drift, and timeline visualizations.
Voice	Browser SpeechSynthesis API	Matches MVP voice requirement without custom infrastructure.
AI layer	Optional LLM adapter service	Allows mock agent responses first; model integration later.
Testing	Vitest + React Testing Library	Unit tests for pure functions and UI behavior.
Deployment	Vercel	Best fit for Next.js class/demo MVP.

MVP architecture decision

Start without Supabase, authentication, or live APIs. Build the engine with seeded data first. After the reactive loop is proven, persistence can be added without rewriting the core services if interfaces are respected.

3. Repository and Folder Structure

The folder structure should make it impossible for Cursor to mix UI, simulation, business logic, and agent explanation responsibilities.

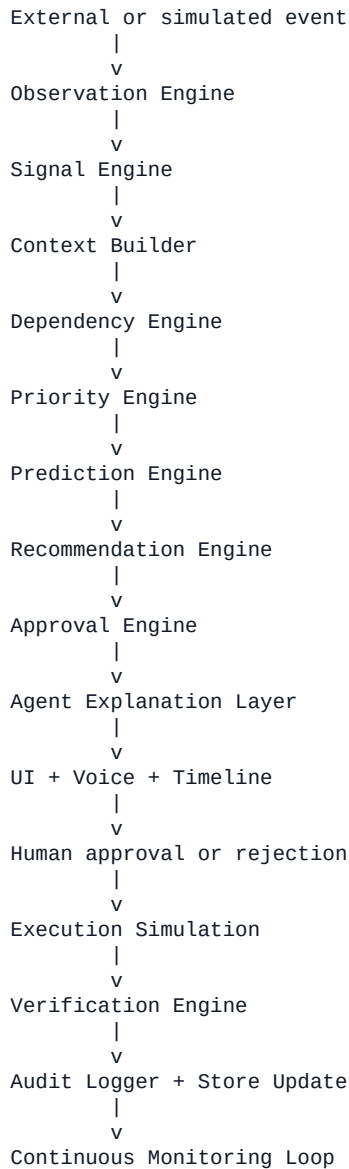
```
pulsebuild/  
  app/  
    page.tsx  
    layout.tsx  
    globals.css  
  components/  
    command-center/  
    cards/  
    chat/  
    timeline/  
    approvals/  
    verification/  
    layout/  
  domain/  
    types/  
    rules/  
    constants/  
  data/  
    seedProject.ts  
    seedTasks.ts  
    seedCrews.ts  
    seedMaterials.ts  
    seedEquipment.ts  
    seedInspections.ts  
    seedPermits.ts  
    seedWeather.ts  
    seedSignals.ts  
  engines/  
    observationEngine.ts  
    signalEngine.ts  
    priorityEngine.ts  
    dependencyEngine.ts  
    predictionEngine.ts  
    recommendationEngine.ts  
    approvalEngine.ts  
    verificationEngine.ts  
    healthEngine.ts  
    driftEngine.ts  
  agent/  
    agentOrchestrator.ts  
    chatAdapter.ts  
    voiceAdapter.ts  
    explanationBuilder.ts  
    promptTemplates.ts  
  simulation/  
    simulationRunner.ts  
    simulationEvents.ts  
    simulationClock.ts  
  store/  
    useProjectStore.ts  
    useSimulationStore.ts  
    useAgentStore.ts  
  lib/  
    eventBus.ts  
    auditLogger.ts  
    formatters.ts  
    ids.ts  
  tests/  
    engines/  
    components/  
    integration/  
  docs/  
    source-of-truth.pdf  
    cognitive-reasoning-spec.pdf  
    technical-architecture.pdf
```

Folder ownership rules

- components/ renders user interface only. Components can call store actions, but cannot calculate priority, health, drift, or recommendations.
- engines/ contains pure functions and deterministic business logic. These functions must be testable without React.
- agent/ explains and orchestrates. It does not own project truth.
- simulation/ creates fake events for the MVP. It should never be imported by core business engines.
- data/ stores seed objects. Seed data should be typed and reusable.
- store/ contains app state and dispatches engine outputs to the UI.

4. System Runtime Architecture

At runtime, PulseBuild behaves like a reflex system. A new signal enters the system, passes through deterministic engines, creates a recommendation, waits for approval when needed, verifies the outcome, logs everything, and updates the Command Center.



Runtime invariant

A signal should never jump directly from detection to UI without being normalized, prioritized, logged, and connected to project context. A recommendation should never appear without evidence, confidence, approval status, and a verification plan.

5. Domain Model and TypeScript Contracts

Cursor should create TypeScript interfaces before implementing UI. Interfaces define the language of PulseBuild and keep every module consistent.

```
export type PriorityTier = 'Tier1' | 'Tier2' | 'Tier3' | 'Tier4';
export type SignalStatus =
  | 'detected'
  | 'validated'
  | 'prioritized'
  | 'recommendation_generated'
  | 'awaiting_approval'
  | 'action_taken'
  | 'verification_pending'
  | 'resolved'
  | 'archived';

export type TaskState =
  | 'planned'
  | 'ready'
  | 'waiting'
  | 'blocked'
  | 'executing'
  | 'paused'
  | 'completed'
  | 'verified'
  | 'closed';

export interface ProjectSignal {
  id: string;
  projectId: string;
  type: SignalType;
  status: SignalStatus;
  createdAt: string;
  updatedAt: string;
  source: 'simulation' | 'user' | 'system';
  relatedEntityId?: string;
  relatedTaskId?: string;
  severityHint?: PriorityTier;
  confidence: number;
  rawData: Record<string, unknown>;
}

export interface EngineResult<T> {
  ok: boolean;
  data: T;
  warnings: string[];
  confidence: number;
  evidence: EvidenceItem[];
}

export interface Recommendation {
  id: string;
  signalId: string;
  intent: OperationalIntent;
  action: string;
  rationale: string;
  expectedBenefit: string;
  tradeoffs: string[];
  confidence: number;
  requiresApproval: boolean;
  approvalReason?: string;
  verificationPlan: VerificationPlan;
  alternatives: RecommendationAlternative[];
}
```

Contract rule

All engines return structured data, not UI strings. Human-readable language should be generated by the explanation builder using structured engine results.

6. Seeded Data Strategy

Seeded data must be realistic enough to prove the reactive loop. The MVP should use one construction project with enough tasks and dependencies to demonstrate cascade reasoning.

Seed file	Minimum contents	Purpose
seedProject.ts	1 project with phase, baseline schedule, current date	Sets the demo context.
seedTasks.ts	12-20 tasks across sitework, structure, MEP, finishes	Enables dependency chains.
seedCrews.ts	4-6 crews with trade, capacity, assigned tasks	Enables idle labor and crew reassignment.
seedMaterials.ts	Concrete, rebar, drywall, conduit, fixtures	Enables material delay scenarios.
seedEquipment.ts	Crane, lift, excavator, pump truck	Enables equipment failure scenarios.
seedInspections.ts	Slab, framing, electrical, safety inspections	Enables legal/safety blockers.
seedPermits.ts	Electrical, road closure, occupancy-related permits	Enables compliance blockers.
seedWeather.ts	Rain, wind, heat, forecast windows	Enables weather conflict logic.
seedSignals.ts	10-15 scripted events	Enables demo mode.

Seed data quality rules

- Every task must have a unique ID and dependency IDs.
- At least three tasks must be on a critical path.
- At least one Tier 1 and one Tier 2 scenario must demonstrate detect-react-verify.
- Every recommendation must be traceable to seeded data, not invented by chat.
- The demo project should include one scenario that resolves and one scenario that remains unresolved after verification.

7. Reactive Engine Services

Each engine has one responsibility. Cursor should build and test engines in this order before connecting them to the UI.

Engine	Responsibility	Input	Output
Observation Engine	Detect events and create signal candidates	Simulation/user event	ProjectSignal draft
Signal Engine	Normalize signal type and status	ProjectSignal draft	Validated ProjectSignal
Dependency Engine	Find upstream/downstream impacts	Signal + project context	Affected tasks, crews, milestones
Priority Engine	Calculate Tier 1-4 and attention score	Signal + dependency result	Priority result + explanation
Prediction Engine	Estimate consequences if ignored	Signal + dependency + priority	Delay, idle labor, drift, health impact
Recommendation Engine	Generate and rank recovery options	Prediction + context	Primary recommendation + alternatives
Approval Engine	Determine human approval requirement	Recommendation + safety rules	Approval status
Verification Engine	Check whether action improved condition	Action + updated state	Resolved/partial/unresolved
Health Engine	Calculate project health	Signals + tasks + verification	Health score + causes
Drift Engine	Calculate operational drift	Signals + task status	Drift score + causes

Engine implementation pattern

```
export function runPriorityEngine(input: PriorityInput): EngineResult<PriorityOutput> {  
  // 1. validate input  
  // 2. apply deterministic rules  
  // 3. create evidence list  
  // 4. calculate confidence  
  // 5. return typed result  
}
```

8. Agent Orchestration Layer

The Agent Orchestrator coordinates the deterministic engines and prepares outputs for the chat, voice, timeline, and UI. It is not the same as the language model.

Orchestrator responsibilities

- Receive validated signals from the store or event bus.
- Run engines in the correct order.
- Combine engine outputs into a single AgentDecision object.
- Build a structured explanation payload.
- Emit events to timeline and UI state.
- Call voice adapter only for Tier 1 or approved escalation conditions.
- Never skip approval for high-impact actions.
- Never mark a recommendation resolved without verification.

```
export interface AgentDecision {  
  signal: ProjectSignal;  
  priority: PriorityOutput;  
  dependencies: DependencyOutput;  
  prediction: PredictionOutput;  
  recommendation: Recommendation;  
  approval: ApprovalResult;  
  explanation: AgentExplanation;  
  auditEvents: TimelineEvent[];  
}
```

LLM boundary

The LLM can turn AgentDecision into clear language, answer follow-up questions, and summarize evidence. It cannot independently change priority, create recommendation IDs, close signals, approve actions, or rewrite project state.

9. State Management Architecture

PulseBuild state should be centralized and predictable. For MVP, Zustand is a good choice because it is simple, fast, and easy for Cursor to understand.

Store	State held	Primary actions
useProjectStore	project, tasks, crews, materials, equipment, inspections, permits, weather	updateTaskState, updateCrewAssignment, updateMaterialStatus
useSimulationStore	simulation status, speed, current step, event queue	startSimulation, pauseSimulation, emitNextEvent, resetSimulation
useAgentStore	signals, recommendations, approvals, verification records, timeline	ingestSignal, createDecision, approveRecommendation, rejectRecommendation, verifyAction
useUIStore	selected signal, filters, panels, muted voice, modal state	openSignal, closePanel, setFilter, toggleVoiceMute

State update rule

Do not update multiple stores randomly from components. Components should call named actions. Store actions can call engines or orchestrator functions, then commit structured updates.

```
// Good
approveRecommendation(recommendationId)

// Avoid
set({ recommendations: manuallyModifiedArray }) from a component
```

10. User Interface Architecture

The UI must feel alive while remaining simple. The Command Center is the main workspace. Components should be modular and driven by state, not hardcoded demo text.

Component	Purpose	Reads from	Actions
CommandCenterShell	Overall layout	Project + UI stores	None
HealthStrip	Health, drift, Tier 1 count	Agent + project stores	Open score details
LiveSignalFeed	Newest signals	Agent store	Select signal, filter priority
RecommendationQueue	Recovery actions	Agent store	Approve, reject, request alternative
AgentChatPanel	Conversation interface	Agent store	Ask, approve, verify
TimelinePanel	Audit history	Agent store	Open event details
ApprovalDialog	Human control	Agent store	Approve/reject
VerificationCard	Outcome check	Agent store	Run verification, view next action
VoiceStatusBadge	Voice state	UI store	Mute/unmute

UI rules

- All Tier 1 signals must be visually impossible to miss.
- No recommendation card should appear without confidence, evidence, approval status, and verification plan.
- Every score must have a clickable explanation.
- Every simulated event must appear in the timeline.
- Empty states should teach the user what will happen when the simulation starts.

11. Event Bus and Timeline Logging

The event bus is the internal nervous system. It carries events from simulation and engine outputs into stores, UI, timeline, and voice. Timeline logging is mandatory for trust and demo clarity.

Event type	When emitted	Timeline entry
signal.detected	New event enters Observation Engine	Signal detected
signal.prioritized	Priority Engine completes	Priority calculated
dependency.resolved	Dependency Engine completes	Dependencies identified
recommendation.created	Recommendation Engine completes	Recommendation ready
approval.required	Approval Engine blocks action	Approval needed
approval.approved	User approves action	Action approved
approval.rejected	User rejects action	Action rejected
verification.completed	Verification Engine completes	Resolved/partial/unresolved
health.updated	Health Engine recalculates	Health changed
drift.updated	Drift Engine recalculates	Drift changed

Timeline event contract

```
export interface TimelineEvent {
  id: string;
  projectId: string;
  type: TimelineEventType;
  timestamp: string;
  title: string;
  description: string;
  relatedSignalId?: string;
  relatedRecommendationId?: string;
  severity?: PriorityTier;
  evidenceIds: string[];
}
```

12. Simulation Engine Architecture

The simulation engine creates the live MVP experience. It should be deterministic by default so demos are reliable, with optional randomization only after the core workflow works.

Simulation modes

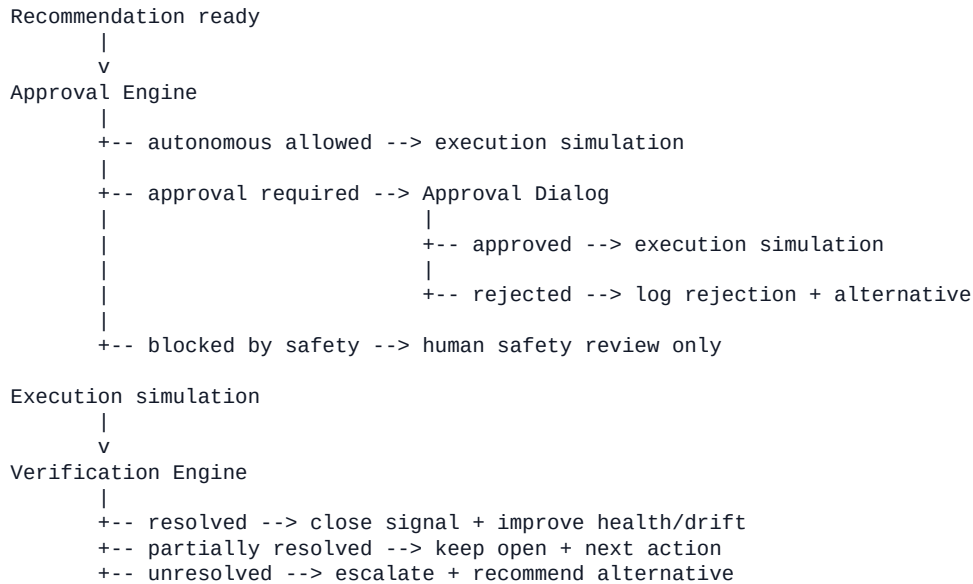
Mode	Behavior	Use case
Scripted demo	Events fire in fixed order	Class presentation and reliable testing
Step mode	User clicks next event	Debugging and controlled walkthrough
Timed mode	Events fire every N seconds	Live command-center feel
Randomized mode	Events selected from valid pool	Optional polish after MVP

Required demo scenarios

- Tier 1 inspection missing before concrete pour: blocks dependent work and requires approval.
- Tier 2 material delay: predicts idle labor and recommends resequencing.
- Tier 2 weather alert: compares weather to outdoor tasks and recommends indoor work.
- Critical equipment failure: recommends backup equipment or task resequencing.
- Resolved verification path: one approved action improves health and closes the loop.
- Unresolved verification path: one action partially fails and produces a next-best action.

13. Approval and Verification Workflow

Approval and verification are what make PulseBuild trustworthy. A reactive agent that recommends but never verifies is incomplete.



Approval rules

- Crew reassignment requires approval.
- Inspection rescheduling requires approval.
- Permit-related changes require approval.
- Safety-related actions are never autonomous.
- Notification-only actions may be automatic.
- Every approval and rejection must be logged with timestamp and reason.

14. Chat and Voice Integration

The chat and voice layer should sit on top of structured agent decisions. It should not replace the reactive engine.

Chat response builder

```
buildAgentResponse(decision):  
  Situation: one sentence summary  
  Priority: tier + reason  
  Evidence: source data only  
  Impact: affected tasks, crews, health, drift  
  Recommendation: next best action  
  Approval: required / not required / blocked  
  Verification: how success will be checked
```

Voice adapter rules

- Use browser SpeechSynthesis API for MVP.
- Speak Tier 1 automatically unless muted.
- Speak Tier 2 only if critical path impact is high or user enables it.
- Never speak Tier 3 or Tier 4 by default.
- No duplicate voice loops for the same signal.
- Voice text must be under 20 seconds and include issue, impact, and required action.

15. API and Persistence Strategy

The MVP can be local-first using seed data and in-memory state. Still, Cursor should write code as if persistence can be added later.

Stage	Persistence approach	Reason
MVP phase 1	Static seed files + Zustand state	Fastest path to working demo.
MVP phase 2	LocalStorage snapshot for demo continuity	Allows reset and reload without backend.
Future phase	Supabase or PostgreSQL	Real user accounts and persistent project records.
Enterprise future	Integrations with Procore/Autodesk/etc.	Out of scope for MVP.

Future API boundaries

```
GET /api/projects/:id
GET /api/projects/:id/signals
POST /api/projects/:id/signals
POST /api/recommendations/:id/approve
POST /api/recommendations/:id/reject
POST /api/recommendations/:id/verify
GET /api/projects/:id/timeline
```

Do not build these API routes until the local MVP works unless Product Owner approves.

16. Security, Guardrails, and Scope Boundaries

Even as a classroom MVP, PulseBuild should behave like a responsible construction decision-support system.

Risk	Architecture guardrail
AI hallucination	LLM only explains structured engine outputs.
Unsafe recommendation	Safety and approval rules run before execution.
Untraceable decisions	Audit Logger records every signal, recommendation, action, and verification.
Scope creep	Cursor must stop if asked to build integrations, auth, BIM, payments, legal decisions, or real automation.
State corruption	Use typed store actions and immutable updates.
Duplicate signals	Signal Engine should detect duplicate IDs or same source/time/type patterns.
Silent failures	EngineResult must return warnings and confidence.

Safety boundary

PulseBuild does not make safety decisions. It detects safety-related conditions, flags them as critical, stops automation, requires human review, and logs the action.

17. Testing Strategy

Testing should focus on the deterministic engines first because these define whether PulseBuild behaves correctly.

Test type	What to test	Examples
Unit tests	Pure engine functions	Priority tier, dependency traversal, health score, drift score
Integration tests	Full signal-to-recommendation pipeline	Material delay produces recommendation with approval status
UI tests	Command Center interactions	Signal appears, recommendation approves, verification updates
Simulation tests	Scripted demo reliability	Events fire in correct order and create timeline entries
Guardrail tests	Forbidden behavior	Safety signal cannot execute automatically
Regression tests	Known scenarios	Inspection missing, weather alert, crew shortage

Minimum tests before demo

- Priority Engine classifies seeded Tier 1 and Tier 2 signals correctly.
- Dependency Engine returns correct downstream tasks for at least three chains.
- Health and Drift scores update after signal and after verification.
- Recommendation Engine produces a primary action for every Tier 1 and Tier 2 signal.
- Approval Engine blocks high-impact actions until approved.
- Verification Engine can produce resolved and unresolved outcomes.
- Command Center answers top blocker in under five seconds.

18. Cursor Build Workflow

Cursor should be treated like an implementation assistant, not a product owner. It must follow the two source documents and this architecture document.

Cursor working rules

- Before coding, Cursor must summarize the requested phase and identify files it plans to create or modify.
- Cursor must not create features outside the approved phase.
- Cursor must ask before changing architecture, stack, scope, or agent behavior.
- Cursor must build services and tests before polishing UI.
- Cursor must keep code readable for a class project and avoid unnecessary complexity.
- Cursor must preserve deterministic-first architecture.
- Cursor must not connect real APIs unless explicitly approved.

Recommended Cursor routine

1. Upload three documents:
 - PulseBuild Source of Truth v2.0
 - PulseBuild Cognitive & Reasoning Specification
 - PulseBuild Technical Architecture v1.0
2. Ask Cursor to read all three and summarize scope.
3. Ask Cursor to create a Phase 1 plan only.
4. Approve the plan.
5. Build one phase at a time.
6. After each phase, ask Cursor for files changed, features completed, and next risks.

19. Phase-by-Phase Implementation Plan

Phase	Goal	Deliverables	Do not build yet
1 Foundation	Create app shell and design tokens	Next.js, Tailwind, layout, navigation, empty Command Center	Engines, AI, backend
2 Data	Create typed seeded project data	Project, tasks, dependencies, crews, materials, equipment, inspections	Live simulation
3 Engines	Build deterministic core	Priority, dependency, health, drift, recommendation basics	Chat/voice polish
4 Command Center	Connect UI to engine state	Live feed, health strip, recommendation queue, timeline	External APIs
5 Simulation	Make app feel reactive	Scripted events, timed mode, reset demo	Random complex scenarios
6 Approval + Verification	Close the reactive loop	Approval dialog, execution simulation, verification outcomes	Real automation
7 Chat + Voice	Add conversational layer	Structured chat responses, Tier 1 voice alerts	General assistant scope
8 Polish	Demo-ready experience	Animations, responsive layout, edge states, testing	Enterprise features

Phase gate rule

A phase is complete only when the app runs, the relevant UI is visible, state updates correctly, and Cursor can explain how the implementation maps to the source documents.

20. Definition of Done

PulseBuild MVP is technically ready when it demonstrates one full operational reflex loop with a professional interface and traceable logic.

MVP technical done checklist

- Application opens to Command Center without errors.
- Seeded project data loads and renders.
- Simulation can start, pause, reset, and emit events.
- New signal appears automatically in Live Signal Feed.
- Signal is classified and prioritized in under one second in the browser.
- Dependency chain is shown for affected tasks.
- Health and Drift scores update after signal processing.
- Recommendation appears for every Tier 1 and Tier 2 signal.
- High-impact recommendation requires approval.
- Approved action creates timeline event and triggers verification.
- Verification can resolve or keep issue open with next action.
- Agent chat answers what is blocking work using project data.
- Tier 1 voice announcement works and can be muted.
- Audit timeline contains signal, analysis, recommendation, approval, verification.
- No out-of-scope integrations or unsafe automations are present.

21. Cursor Starter Prompt

Use this prompt when you begin building in Cursor after uploading the three documents.

You are helping me build PulseBuild, a reactive construction operations agent MVP.

Read these three governing documents before planning or coding:

1. PulseBuild Source of Truth v2.0
2. PulseBuild Cognitive & Reasoning Specification
3. PulseBuild Technical Architecture v1.0

Important rules:

- Do not invent scope.
- Do not build integrations, authentication, real sensors, or enterprise features.
- Build deterministic business logic before AI polish.
- Keep UI, engines, simulation, agent explanation, and state management separate.
- The LLM explains structured engine outputs; it does not calculate project truth.
- Every Tier 1 and Tier 2 signal must generate a recommendation.
- High-impact actions require approval.
- Every executed action must be verified.
- Every signal, recommendation, approval, rejection, and verification must be logged.

Start with Phase 1 only:

Create the Next.js TypeScript project structure, Tailwind setup, design tokens, app shell, Command Center layout,

Final architecture note

Do not wait for perfect documentation. These three documents are sufficient to begin. Any future gaps should be captured as small architectural decisions while building.